

[Day 2] 종합 실습 결과 정리

판교 5반 홍은비

종합실습 1+ CampusHub— 복합 쿼리 실습 (JOIN + 서브쿼리 + 윈도우함수)

1. 학생과 수강을 INNER JOIN하여 수강 존재 학생의 과목/성적을 조회

--- [종합실습 2 - 1번]
--- 1. 학생과 수강을 INNER JOIN하여 수강 존재 학생의 과목/성적을 조회

--- INNER JOIN 특징:
--- student와 enroll 양쪽에 student_id가 존재하는 행만 출력된다.
--- 따라서 수강 기록이 없는 학생과 학생 정보가 없는 고이 수강은 제외된다.

```

SELECT
  s.student_id AS 학생_ID, -- 학생의 고유번호
  s.name AS 학생명, -- 학생 이름
  e.course AS 과목, -- 수강 과목명
  e.grade AS 성적 -- 해당 과목의 성적
FROM lab.student AS s
INNER JOIN lab.enroll AS e
  ON s.student_id = e.student_id
  -- 두 테이블에서 student_id가 같은 행끼리 연결한다.
  -- INNER JOIN이므로 양쪽 테이블에 모두 존재하는 행만 남는다.
ORDER BY
  s.student_id ASC, -- 학생 번호를 오름차순으로 정렬
  e.course ASC; -- 같은 학생은 과목명을 오름차순으로 정렬
  
```

student_id	학생_ID	학생명	과목	성적
2282	992	Student_992	Course_15	C
2283	993	Student_993	Course_15	C
2284	993	Student_993	Course_16	D
2285	993	Student_993	Course_17	A
2286	994	Student_994	Course_16	D
2287	994	Student_994	Course_17	A
2288	995	Student_995	Course_17	A
2289	995	Student_995	Course_18	B
2290	995	Student_995	Course_19	C
2291	996	Student_996	Course_18	B
2292	996	Student_996	Course_19	C
2293	997	Student_997	Course_1	A
2294	997	Student_997	Course_19	C
2295	997	Student_997	Course_20	D
2296	998	Student_998	Course_1	A
2297	998	Student_998	Course_20	D
2298	999	Student_999	Course_1	A
2299	999	Student_999	Course_2	B
2300	999	Student_999	Course_3	C

2. 모든 학생 기준으로 수강을 붙이고, 과목(없으면 NULL)까지 보이기

--- [종합실습 2 - 2번]
--- 모든 학생을 기준으로 수강 정보를 연결한다.

--- LEFT JOIN을 사용하므로 수강 기록이 없는 학생도 출력된다.
--- 수강 기록이 없으면 과목과 성적이 NULL로 표시된다.

```

SELECT
  s.student_id AS 학생_ID, -- 학생 번호
  s.name AS 학생명, -- 학생 이름
  s.major AS 전공, -- 학생 전공
  e.course AS 과목, -- 수강 과목, 미수강이면 NULL
  e.grade AS 성적 -- 과목 성적, 미수강이면 NULL
FROM lab.student AS s
LEFT JOIN lab.enroll AS e
  ON s.student_id = e.student_id
  -- 수강 정보가 없어도 왼쪽 student 행은 유지된다.
ORDER BY
  s.student_id, -- 학생 번호 오름차순
  e.course; -- 같은 학생은 과목명 오름차순
  
```

student_id	학생_ID	학생명	전공	과목	성적
2381	992	Student_992	HR	Course_15	C
2382	993	Student_993	HR	Course_15	C
2383	993	Student_993	HR	Course_16	D
2384	993	Student_993	HR	Course_17	A
2385	994	Student_994	HR	Course_16	D
2386	994	Student_994	HR	Course_17	A
2387	995	Student_995	HR	Course_17	A
2388	995	Student_995	HR	Course_18	B
2389	995	Student_995	HR	Course_19	C
2390	996	Student_996	HR	Course_18	B
2391	996	Student_996	HR	Course_19	C
2392	997	Student_997	HR	Course_1	A
2393	997	Student_997	HR	Course_19	C
2394	997	Student_997	HR	Course_20	D
2395	998	Student_998	HR	Course_1	A
2396	998	Student_998	HR	Course_20	D
2397	999	Student_999	HR	Course_1	A
2398	999	Student_999	HR	Course_2	B
2399	999	Student_999	HR	Course_3	C
2400	1000	Student_1000	HR	[NULL]	[NULL]

- LEFT JOIN은 왼쪽에 있는 student를 기준으로 모든 데이터를 유지 → student에 없는 1001, 1010번은 출력되지 않음

3. 수강이 기준. 학생이 없으면 학생 정보가 NULL

--- [종합실습 2 - 3번]
--- 모든 수강 기록을 기준으로 학생 정보를 연결한다.
--- RIGHT JOIN을 사용하므로 student에 없는 수강 기록도 출력된다.
--- 학생 정보가 없는 수강 기록은 학생_ID, 학생명, 전공이 NULL로 표시된다.

```

SELECT
  s.student_id AS 학생_ID,      -- student에 존재하지 않으면 NULL
  s.name AS 학생명,            -- student에 존재하지 않으면 NULL
  s.major AS 전공,             -- student에 존재하지 않으면 NULL
  e.student_id AS 수강_학생_ID, -- enroll에 기록된 학생 번호
  e.course AS 과목,             -- 수강 과목
  e.grade AS 성적              -- 과목 성적
FROM lab.student AS s
RIGHT JOIN lab.enroll AS e
  ON s.student_id = e.student_id
  -- student에 같은 학생 번호가 없어도 enroll 행은 출력된다.
ORDER BY
  e.student_id,
  e.course;

```

123	학생_id	AZ 학생명	AZ 전공	123 수강_학생_id	AZ 과목	AZ 성적
2284	993	Student_993	HR	993	Course_16	D
2285	993	Student_993	HR	993	Course_17	A
2286	994	Student_994	HR	994	Course_16	D
2287	994	Student_994	HR	994	Course_17	A
2288	995	Student_995	HR	995	Course_17	A
2289	995	Student_995	HR	995	Course_18	B
2290	995	Student_995	HR	995	Course_19	C
2291	996	Student_996	HR	996	Course_18	B
2292	996	Student_996	HR	996	Course_19	C
2293	997	Student_997	HR	997	Course_1	A
2294	997	Student_997	HR	997	Course_19	C
2295	997	Student_997	HR	997	Course_20	D
2296	998	Student_998	HR	998	Course_1	A
2297	998	Student_998	HR	998	Course_20	D
2298	999	Student_999	HR	999	Course_1	A
2299	999	Student_999	HR	999	Course_2	B
2300	999	Student_999	HR	999	Course_3	C
2301	[NULL]	[NULL]	[NULL]	1001	AI	A
2302	[NULL]	[NULL]	[NULL]	1010	ML	B

- RIGHT JOIN은 오른쪽에 있는 enroll을 기준으로 모든 데이터를 유지
→ 학생 정보가 없는 1001, 1010번 수강도 출력 (이때 학생 정보는 NULL)

4. 학생/수강 모두 포함

--- [종합실습 2 - 4번]
--- FULL OUTER JOIN으로 학생과 수강 양쪽의 모든 데이터를 조회한다.
--- 포함되는 데이터:
--- 1. 학생 정보와 수강 정보가 모두 있는 정상 수강 기록
--- 2. 학생 정보만 있는 미수강 학생
--- 3. 수강 정보만 있는 고아 수강 기록(1001, 1010)

```

SELECT
  s.student_id AS 학생_ID,      -- student의 학생 번호
  s.name AS 학생명,            -- 학생 정보가 없으면 NULL
  s.major AS 전공,             -- 학생 정보가 없으면 NULL
  e.student_id AS 수강_학생_ID, -- enroll에 기록된 학생 번호
  e.course AS 과목,             -- 미수강 학생이면 NULL
  e.grade AS 성적              -- 미수강 학생이면 NULL
FROM lab.student AS s
FULL OUTER JOIN lab.enroll AS e
  ON s.student_id = e.student_id
ORDER BY
  COALESCE(s.student_id, e.student_id) ASC,
  e.course ASC;

```

123	학생_id	AZ 학생명	AZ 전공	123 수강_학생_id	AZ 과목	AZ 성적
2386	994	Student_994	HR	994	Course_17	A
2387	995	Student_995	HR	995	Course_17	A
2388	995	Student_995	HR	995	Course_18	B
2389	995	Student_995	HR	995	Course_19	C
2390	996	Student_996	HR	996	Course_18	B
2391	996	Student_996	HR	996	Course_19	C
2392	997	Student_997	HR	997	Course_1	A
2393	997	Student_997	HR	997	Course_19	C
2394	997	Student_997	HR	997	Course_20	D
2395	998	Student_998	HR	998	Course_1	A
2396	998	Student_998	HR	998	Course_20	D
2397	999	Student_999	HR	999	Course_1	A
2398	999	Student_999	HR	999	Course_2	B
2399	999	Student_999	HR	999	Course_3	C
2400	1000	Student_1000	HR	[NULL]	[NULL]	[NULL]
2401	[NULL]	[NULL]	[NULL]	1001	AI	A
2402	[NULL]	[NULL]	[NULL]	1010	ML	B

5. 한 번도 수강하지 않은 학생 목록

SQL

Commit

Rollback

Auto

skala_db lab@skala_db

Script-2 X

=====

-- [종합실습 2 - 5번]

-- 한 번도 수강하지 않은 학생 목록을 조회한다.

--

-- 처리 방식:

-- 1. LEFT JOIN으로 모든 학생을 유지한다.

-- 2. 수강 기록이 없는 학생은 enroll 컬럼이 NULL이 된다.

-- 3. WHERE에서 enroll의 student_id가 NULL인 행만 선택한다.

-- 이 방식을 LEFT JOIN 기반 Anti Join이라고 한다.

=====

SELECT

s.student_id AS 학생_ID, -- 학생 번호

s.name AS 학생명, -- 학생 이름

s.major AS 전공, -- 학생 전공

s.gpa AS GPA -- 학생 GPA

FROM lab.student AS s

LEFT JOIN lab.enroll AS e

ON s.student_id = e.student_id

WHERE e.student_id IS NULL -- 수강 기록이 없는 학생만 선택

ORDER BY s.student_id ASC;

student 1 X

SELECT s.student_id AS 학생_ID, s.name AS 학생명

Enter a SQL expression to filter results (use Ctrl+Space)

123 학생_ID

AZ 학생명

AZ 전공

123 gpa

	123 학생_ID	AZ 학생명	AZ 전공	123 gpa
83	830	Student_830	CS	4
84	840	Student_840	CS	2
85	850	Student_850	CS	3
86	860	Student_860	CS	4
87	870	Student_870	CS	2
88	880	Student_880	CS	3
89	890	Student_890	CS	4
90	900	Student_900	CS	2
91	910	Student_910	CS	3
92	920	Student_920	CS	4
93	930	Student_930	CS	2
94	940	Student_940	CS	3
95	950	Student_950	CS	4
96	960	Student_960	CS	2
97	970	Student_970	CS	3
98	980	Student_980	CS	4
99	990	Student_990	HR	2
100	1,000	Student_1000	HR	3

Value X

10

Refresh

Save

Cancel

Export data

200

100

100 row(s) fetched - 0.003s, on 2026-08-12 at 15:50:08

- 쿼리의 기준이 student이라서 고아 수강 1001, 1010은 포함X

6. 한 과목 이상 수강한 학생 목록(중복 제거)

SQL

Commit

Rollback

Auto

skala_db lab@skala_db

Script-2 X

=====

-- [종합실습 2 - 6번]

-- 한 과목 이상 수강한 학생 목록을 조회한다.

--

-- 처리 방식:

-- 1. INNER JOIN으로 학생과 수강 기록을 연결한다.

-- 2. 한 학생이 여러 과목을 수강하면 같은 학생이 여러 행으로 나온다.

-- 3. DISTINCT로 중복 학생을 제거해 학생당 한 행만 출력한다.

=====

SELECT DISTINCT

s.student_id AS 학생_ID, -- 학생 번호

s.name AS 학생명, -- 학생 이름

s.major AS 전공, -- 학생 전공

s.gpa AS GPA -- 학생 GPA

FROM lab.student AS s

INNER JOIN lab.enroll AS e

ON s.student_id = e.student_id

-- 수강 기록이 존재하는 학생만 연결된다.

ORDER BY s.student_id ASC;

student 1 X

SELECT DISTINCT s.student_id AS 학생_ID, s.name AS 학생명

Enter a SQL expression to filter results (use Ctrl+Space)

123 학생_ID

AZ 학생명

AZ 전공

123 gpa

	123 학생_ID	AZ 학생명	AZ 전공	123 gpa
881	978	Student_978	CE	3.8
882	979	Student_979	BIO	3.9
883	981	Student_981	HR	4.1
884	982	Student_982	HR	4.2
885	983	Student_983	HR	4.3
886	984	Student_984	HR	4.4
887	985	Student_985	HR	4.5
888	986	Student_986	HR	4.6
889	987	Student_987	HR	4.7
890	988	Student_988	HR	4.8
891	989	Student_989	HR	4.9
892	991	Student_991	HR	2.1
893	992	Student_992	HR	2.2
894	993	Student_993	HR	2.3
895	994	Student_994	HR	2.4
896	995	Student_995	HR	2.5
897	996	Student_996	HR	2.6
898	997	Student_997	HR	2.7
899	998	Student_998	HR	2.8
900	999	Student_999	HR	2.9

Value X

1

Refresh

Save

Cancel

Export data

200

900

900 row(s) fetched - 0.002s, on 2026-08-12 at 15:52:27

7. 고객별 주문건수/총액

*skala_db> Script-2 X

```
-- [종합실습 2 - 7번]
-- 고객별 주문 건수와 주문 총액을 조회한다.
--
-- 처리 방식:
-- 1. customers와 orders를 customer_id로 연결한다.
-- 2. 고객별로 GROUP BY한다.
-- 3. COUNT로 주문 건수를 계산한다.
-- 4. SUM으로 주문 총액을 계산한다.
--
```

```
SELECT
  c.customer_id AS 고객_ID,    -- 고객 번호
  c.customer_name AS 고객명,   -- 고객 이름
  COUNT(o.order_id) AS 주문_건수, -- 고객별 주문 개수
  SUM(o.amount) AS 주문_총액   -- 고객별 주문금액 합계
FROM lab.customers AS c
INNER JOIN lab.orders AS o
  ON c.customer_id = o.customer_id
  -- customers와 orders의 고객 번호가 같은 행을 연결한다.
GROUP BY
  c.customer_id,
  c.customer_name
  -- 고객 한 명을 하나의 그룹으로 묶는다.
ORDER BY
  c.customer_id ASC;
```

customers 1 X

SELECT c.customer_id AS 고객_ID, c.customer_name AS 고객명, COUNT(o.order_id) AS 주문_건수, SUM(o.amount) AS 주문_총액

123	고객_ID	AZ 고객명	123 주문_건수	123 주문_총액
486	486	Customer_486	6	15,365.1
487	487	Customer_487	6	15,443.16
488	488	Customer_488	6	15,521.22
489	489	Customer_489	6	15,599.28
490	490	Customer_490	6	15,677.34
491	491	Customer_491	6	15,755.4
492	492	Customer_492	6	15,833.46
493	493	Customer_493	6	15,911.52
494	494	Customer_494	6	15,989.58
495	495	Customer_495	6	16,067.64
496	496	Customer_496	6	16,145.7
497	497	Customer_497	6	16,223.76
498	498	Customer_498	6	16,301.82
499	499	Customer_499	6	16,379.88
500	500	Customer_500	6	16,457.94

8. 총액 상위 10명과 금액

*skala_db> Script-2 X

```
-- [종합실습 2 - 8번]
-- 고객별 주문 총액을 계산하고,
-- 주문 총액이 높은 고객 상위 10명을 조회한다.
--
-- 처리 방식:
-- 1. customers와 orders를 customer_id로 연결한다.
-- 2. 고객별로 주문금액을 합산한다.
-- 3. 주문 총액을 내림차순으로 정렬한다.
-- 4. LIMIT 10으로 상위 10명만 출력한다.
--
```

```
SELECT
  c.customer_id AS 고객_ID,    -- 고객 번호
  c.customer_name AS 고객명,   -- 고객 이름
  SUM(o.amount) AS 주문_총액   -- 고객별 주문금액 합계
FROM lab.customers AS c
INNER JOIN lab.orders AS o
  ON c.customer_id = o.customer_id
  -- 고객 번호가 같은 주문 기록을 연결한다.
GROUP BY
  c.customer_id,
  c.customer_name
  -- 고객 한 명을 하나의 그룹으로 묶는다.
ORDER BY
  주문_총액 DESC, -- 주문 총액이 높은 고객부터 정렬
  c.customer_id ASC -- 총액이 같으면 고객 번호 오름차순
LIMIT 10;
```

customers 1 X Statistics 1

SELECT c.customer_id AS 고객_ID, c.customer_name AS 고객명, SUM(o.amount) AS 주문_총액

123	고객_ID	AZ 고객명	123 주문_총액
1	154	Customer_154	19,467.18
2	153	Customer_153	19,389.12
3	152	Customer_152	19,311.06
4	151	Customer_151	19,233
5	150	Customer_150	19,154.94
6	149	Customer_149	19,076.88
7	148	Customer_148	18,998.82
8	147	Customer_147	18,920.76
9	146	Customer_146	18,842.7
10	145	Customer_145	18,764.64

9. 모든 직원과 그 매니저 이름

*skala_db> Script-2 X

```

-- [종합실습 2 - 9번]
-- 모든 직원과 각 직원의 매니저 이름을 조회한다.
--
-- Self Join:
-- 하나의 emp 테이블을 직원용(e)과 매니저용(m)으로 두 번 참조한다.
--
-- LEFT JOIN을 사용하는 이유:
-- CEO는 manager_id가 NULL이지만 결과에 포함해야 하기 때문이다.
--
=====
SELECT
  e.emp_id AS 직원_ID,      -- 직원 번호
  e.name AS 직원명,        -- 직원 이름
  e.manager_id AS 매니저_ID, -- 직원에게 지정된 매니저 번호
  m.name AS 매니저명,      -- 같은 테이블에서 찾은 매니저 이름
FROM lab.emp AS e
LEFT JOIN lab.emp AS m
  ON e.manager_id = m.emp_id
  -- e: 직원으로 사용한 emp 테이블
  -- m: 매니저로 사용한 동일한 emp 테이블
-- 직원의 manager_id와 매니저의 emp_id를 연결한다.
ORDER BY
  e.emp_id ASC;
=====

```

emp 1 X

SELECT e.emp_id AS 직원_ID, e.name AS 직원명, | Enter a SQL expression to filter results (use Ctrl+Space)

	123 < 직원_ID	AZ < 직원명	123 < 매니저_ID	AZ < 매니저명
294	294	Dev_294	3	Mgr_3
295	295	Dev_295	4	Mgr_4
296	296	Dev_296	5	Mgr_5
297	297	Dev_297	6	Mgr_6
298	298	Dev_298	7	Mgr_7
299	299	Dev_299	8	Mgr_8
300	300	Dev_300	9	Mgr_9
301	301	Dev_301	10	Mgr_10
302	302	Dev_302	1	CEO
303	303	Dev_303	2	Mgr_2
304	304	Dev_304	3	Mgr_3
305	305	Dev_305	4	Mgr_4
306	306	Dev_306	5	Mgr_5
307	307	Dev_307	6	Mgr_6
308	308	Dev_308	7	Mgr_7
309	309	Dev_309	8	Mgr_8
310	310	Dev_310	9	Mgr_9
311	311	Dev_311	10	Mgr_10

Value X

1

10. "모든 학생 기준"으로 과목 분포를 보고 싶다 → LEFT JOIN + 집계

*skala_db> Script-2 X

```

-- [종합실습 2 - 10번]
-- 모든 학생을 기준으로 과목별 학생 수를 집계한다.
--
-- 처리 방식:
-- 1. LEFT JOIN으로 미수강 학생까지 유지한다.
-- 2. 수강 과목이 NULL이면 '미수강'으로 표시한다.
-- 3. 과목별로 GROUP BY하여 학생 수를 계산한다.
--
-- COUNT(s.student_id)를 사용하는 이유:
-- 미수강 학생도 student_id는 존재하므로 100명으로 집계된다.
-- COUNT(e.student_id)를 사용하면 미수강 그룹은 0명으로 계산된다.
--
=====
SELECT
  COALESCE(e.course, '미수강') AS 과목, -- NULL 과목은 미수강으로 표시
  COUNT(s.student_id) AS 학생_수 -- 해당 과목에 포함된 학생 수
FROM lab.student AS s
LEFT JOIN lab.enroll AS e
  ON s.student_id = e.student_id
  -- 모든 학생을 유지하면서 수강 정보를 연결한다.
GROUP BY
  e.course
  -- 과목별로 그룹화하여 NULL도 하나의 그룹으로 집계된다.
ORDER BY
  CASE WHEN e.course IS NULL THEN 1 ELSE 0 END,
  -- 실제 과목을 먼저 표시하고 미수강은 마지막에 표시한다.
  e.course ASC;
=====

```

student(+) 1 X

SELECT COALESCE(e.course, '미수강') AS 과목, | Enter a SQL expression to filter results (use Ctrl+Space)

	AZ < 과목	123 < 학생_수
9	Course_17	144
10	Course_18	96
11	Course_19	144
12	Course_2	47
13	Course_20	96
14	Course_3	94
15	Course_4	94
16	Course_5	141
17	Course_6	94
18	Course_7	141
19	Course_8	94
20	Course_9	144
21	DB	107
22	미수강	100

Value X

Text

Course_1

Refresh Save Cancel Export data 200 22 22 row(s) fetched - 0.004s, on 2026-08-12 at 16:01:59

11. DB 과목을 듣지 않은 모든 학생을 나열

```
SELECT
s.student_id AS 학생_ID, -- 학생 번호
s.name AS 학생명, -- 학생 이름
s.major AS 전공, -- 학생 전공
s.gpa AS GPA -- 학생 GPA
FROM lab.student AS s
WHERE NOT EXISTS (
SELECT 1
FROM lab.enroll AS e
WHERE e.student_id = s.student_id
AND e.course = 'DB'
)
ORDER BY
s.student_id ASC;
```

학생_ID	학생명	전공	GPA
880	Student_987	HR	4.7
881	Student_988	HR	4.8
882	Student_989	HR	4.9
883	Student_990	HR	2
884	Student_991	HR	2.1
885	Student_992	HR	2.2
886	Student_993	HR	2.3
887	Student_994	HR	2.4
888	Student_995	HR	2.5
889	Student_996	HR	2.6
890	Student_997	HR	2.7
891	Student_998	HR	2.8
892	Student_999	HR	2.9
893	Student_1000	HR	3

12. (가정) 과목별로 매니저가 운영 책임을 갖는다고 가정하고,emp의 매니저(이름 'Mgr_'로 시작와 과목을 임의로 매핑한 테이블 course_owner(course, manager_id)를 만든 뒤,과목별 수강 인원 + 책임 매니저 이름 리포트를 작성하세요

```
CREATE TABLE IF NOT EXISTS lab.course_owner (
course VARCHAR(50) PRIMARY KEY,
manager_id INT NOT NULL
);
```

Name	Value
Updated Rows	0
Execute time	0.005s
Start time	Wed Aug 12 16:15:56 KST 2026
Finish time	Wed Aug 12 16:15:56 KST 2026
Query	CREATE TABLE IF NOT EXISTS lab.course_owner (course VARCHAR(50) PRIMARY KEY, manager_id INT NOT NULL);

12.1 빈 course_owner 테이블 생성

```
INSERT INTO lab.course_owner (
course,
manager_id
)
SELECT
e.course,
e.emp_id
FROM lab.enroll AS e
GROUP BY e.course;
```

Name	Value
Updated Rows	23
Execute time	0.005s
Start time	Wed Aug 12 16:20:36 KST 2026
Finish time	Wed Aug 12 16:20:36 KST 2026
Query	INSERT INTO lab.course_owner (course, manager_id) SELECT e.course, e.emp_id FROM lab.enroll AS e GROUP BY e.course;

12.2 해당 테이블에 과목-매니저 23개 매핑 저장

```
SELECT
co.course AS 과목, COUNT(DISTINCT e.student_id) AS 수강_인원, -- 과목명
e.name AS 과목_매니저_ID, -- 과목 매니저 ID
e.name AS 과목_매니저명 -- 과목 매니저 이름
FROM lab.course_owner AS co
LEFT JOIN lab.enroll AS e
ON co.course = e.course
INNER JOIN lab.emp AS e
ON co.manager_id = e.emp_id
GROUP BY
co.course,
e.name
ORDER BY
co.course ASC;
```

과목	수강_인원	과목_매니저_ID	과목_매니저명
Course_14	96	8	Mgr_8
Course_15	144	9	Mgr_9
Course_16	96	10	Mgr_10
Course_17	144	11	Mgr_11
Course_18	96	2	Mgr_2
Course_19	144	3	Mgr_3
Course_2	47	4	Mgr_4
Course_20	96	5	Mgr_5
Course_3	94	6	Mgr_6
Course_4	94	7	Mgr_7
Course_5	141	8	Mgr_8
Course_6	94	9	Mgr_9
Course_7	141	10	Mgr_10
Course_8	94	11	Mgr_11
Course_9	144	2	Mgr_2
DB	107	3	Mgr_3
ML	1	4	Mgr_4

12.3 저장된 매핑을 수강·직원 테이블과 조인해 최종 조회

13. 학생 × 과목 전체 조합을 만들어 "학생별 과목 추천 후보"를 만들되, 샘플 100건만 본다

*skala_db> Script-2 X

```

-- [종합실습 2 - 13번]
-- 학생과 과목의 전체 조합을 생성하여
-- 학생별 과목 추천 후보를 만든다.
--
-- CROSS JOIN:
-- 모든 학생을 모든 과목과 한 번씩 조합한다.
--
-- student는 1,000명이고 과목은 23개이므로
-- 전체 조합은 1,000 × 23 = 23,000행이다.
-- 문제 요구사항에 따라 그중 100행만 조회한다.

```

```

SELECT
  s.student_id AS 학생_ID, -- 학생 번호
  s.name AS 학생명, -- 학생 이름
  s.major AS 전공, -- 학생 전공
  c.course AS 추천_과목 -- 학생과 조합된 과목
FROM lab.student AS s
CROSS JOIN (
  -- 별도의 과목 테이블이 없으므로
  -- enroll에서 중복되지 않은 과목 목록을 만든다.
  SELECT DISTINCT
    e.course
  FROM lab.enroll AS e
) AS c
ORDER BY
  s.student_id ASC,
  c.course ASC
LIMIT 100;

```

student(+) 1 X

SELECT s.student_id AS 학생_ID, s.name AS 학생명, s.major AS 전공, s.course AS 추천_과목

123	학생_id	AZ 학생명	AZ 전공	AZ 추천_과목
86	4	Student_4	BIO	Course_5
87	4	Student_4	BIO	Course_6
88	4	Student_4	BIO	Course_7
89	4	Student_4	BIO	Course_8
90	4	Student_4	BIO	Course_9
91	4	Student_4	BIO	DB
92	4	Student_4	BIO	ML
93	5	Student_5	CS	AI
94	5	Student_5	CS	Course_1
95	5	Student_5	CS	Course_10
96	5	Student_5	CS	Course_11
97	5	Student_5	CS	Course_12
98	5	Student_5	CS	Course_13
99	5	Student_5	CS	Course_14
100	5	Student_5	CS	Course_15

14. 스칼라 서브쿼리 (SELECT 절) 사용 학생 + 소속 학과명 붙이기

*skala_db> Script-2 X

```

-- [종합실습 2 - 14번]
-- SELECT 절의 스칼라 서브쿼리를 사용하여
-- 각 학생에게 소속 학과명을 붙인다.
--
-- 스칼라 서브쿼리:
-- 실행 결과가 하나의 값만 반환하는 서브쿼리이다.
--
-- 현재 실습 DB에는 별도의 학과 테이블이 없으므로
-- student 테이블에서 현재 학생의 major를 다시 조회한다.
-- student_id는 기본키이므로 서브쿼리는 정확히 한 행만 반환한다.

```

```

SELECT
  s.student_id AS 학생_ID, -- 학생 번호
  s.name AS 학생명, -- 학생 이름
  s.gpa AS GPA, -- 학생 GPA
  (
    SELECT
      s2.major
    FROM lab.student AS s2
    WHERE s2.student_id = s.student_id
  ) AS 소속_학과명
FROM lab.student AS s
ORDER BY
  s.student_id ASC;

```

student 1 X

SELECT s.student_id AS 학생_ID, s.name AS 학생명, s.gpa AS GPA, (SELECT s2.major FROM lab.student AS s2 WHERE s2.student_id = s.student_id) AS 소속_학과명

123	학생_id	AZ 학생명	123 gpa	AZ 소속_학과명
986	986	Student_986	4.6	HR
987	987	Student_987	4.7	HR
988	988	Student_988	4.8	HR
989	989	Student_989	4.9	HR
990	990	Student_990	2	HR
991	991	Student_991	2.1	HR
992	992	Student_992	2.2	HR
993	993	Student_993	2.3	HR
994	994	Student_994	2.4	HR
995	995	Student_995	2.5	HR
996	996	Student_996	2.6	HR
997	997	Student_997	2.7	HR
998	998	Student_998	2.8	HR
999	999	Student_999	2.9	HR
1000	1000	Student_1000	3	HR

15. 평균 GPA 보다 높은 학생 (WHERE 서브쿼리)

The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

```

--=====
-- [종합실습 2 - 15번]
-- 전체 학생의 평균 GPA보다 GPA가 높은 학생을 조회한다.
--
-- WHERE 절의 서브쿼리에서 전체 평균 GPA를 먼저 계산한다.
-- 바깥쪽 쿼리는 각 학생의 GPA를 전체 평균과 비교한다.
--=====

SELECT
s.student_id AS 학생_ID, -- 학생 번호
s.name AS 학생명, -- 학생 이름
s.major AS 전공, -- 학생 전공
s.gpa AS GPA -- 학생 GPA
FROM lab.student AS s
WHERE s.gpa > (
    SELECT
    AVG(s2.gpa)
    FROM lab.student AS s2
    -- 전체 학생의 평균 GPA 하나를 반환한다.
)
ORDER BY
s.gpa DESC, -- GPA가 높은 학생부터 정렬
s.student_id ASC; -- GPA가 같으면 학생 번호 오름차순
  
```

The results pane shows a table with 21 rows of data:

	student_id	s.name	s.major	s.gpa
1	29	Student_29	BIO	4.9
2	59	Student_59	BIO	4.9
3	89	Student_89	BIO	4.9
4	119	Student_119	BIO	4.9
5	149	Student_149	BIO	4.9
6	179	Student_179	BIO	4.9
7	209	Student_209	BIO	4.9
8	239	Student_239	BIO	4.9
9	269	Student_269	BIO	4.9
10	299	Student_299	BIO	4.9
11	329	Student_329	BIO	4.9
12	359	Student_359	BIO	4.9
13	389	Student_389	BIO	4.9
14	419	Student_419	BIO	4.9
15	449	Student_449	BIO	4.9
16	479	Student_479	BIO	4.9
17	509	Student_509	BIO	4.9
18	539	Student_539	BIO	4.9
19	569	Student_569	BIO	4.9
20	599	Student_599	BIO	4.9
21	629	Student_629	BIO	4.9

16. 자신의 학과 평균 GPA보다 높은 학생 (Correlated subquery)

```

--> <kcala_db>Script-2 X
SELECT s.student_id ASC; -- GPA가 같으면 학생 번호 오름차순

-- [종합실습 2 - 16번]
-- 각 학생의 GPA를 자신이 속한 학과의 평균 GPA와 비교하여
-- 학과 평균보다 GPA가 높은 학생을 조회한다.
-- 상관 서브쿼리(Correlated Subquery):
-- 안쪽 서브쿼리에서 비일치 쿼리의 s.major를 참조한다.
-- 따라서 학생마다 해당 학생의 학과 평균을 다시 계산하여 비교한다.

SELECT
    s.student_id AS 학생_ID, -- 학생 번호
    s.name AS 학생명, -- 학생 이름
    s.major AS 전공, -- 학생이 속한 학과
    s.gpa AS GPA -- 학생 GPA
FROM lab.student AS s
WHERE s.gpa > (
    SELECT
        AVG(s2.gpa)
    FROM lab.student AS s2
    WHERE s2.major = s.major
    -- 현재 학생과 같은 학과의 학생들만 평균 계산
)
ORDER BY
    s.major ASC, -- 학과명 오름차순
    s.gpa DESC, -- 학과 안에서 GPA 내림차순
    s.student_id ASC; -- GPA가 같으면 학생 번호 오름차순

```

The screenshot also shows the DBeaver interface with the 'Text' tab selected, displaying the SQL query. Below the query, there's a preview of the results table with columns: student_id, name, major, gpa. The first row shows Student_29, BIO, 4.9.

17. 수강(enroll) 기록이 있는 학생만

*cskala_db> Script-2 X

```

s.student_id AS s; -- GPA/기 일보전 학생 번호 노출자본

-- [종합실습 2 - 17번]
-- 수강(enroll) 기록이 하나 이상 존재하는 학생만 조회한다.
--
-- EXISTS:
-- 현재 학생과 일치하는 수강 기록이 한 행이라도 존재하면 TRUE가 된다.
-- EXISTS는 존재 여부만 확인하므로 학생이 여러 과목을 수강해도
-- 학생 한 명당 결과는 한 행만 출력된다.
=====

SELECT
  s.student_id AS 학생_ID, -- 학생 번호
  s.name AS 학생명, -- 학생 이름
  s.major AS 전공, -- 학생 전공
  s.gpa AS GPA -- 학생 GPA
FROM lab.student AS s
WHERE EXISTS (
  SELECT 1
  FROM lab.enroll AS e
  WHERE e.student_id = s.student_id
  -- 현재 학생의 수강 기록이 존재하는지 확인한다.
)
ORDER BY
  s.student_id ASC;

```

student 1 X

SELECT s.student_id AS 학생_ID, s.name AS 학생명

123 = 학생_ID	AZ 학생명	AZ 전공	123 gpa
1	Student_1	EE	2.1
2	Student_2	ME	2.2
3	Student_3	CE	2.3
4	Student_4	BIO	2.4
5	Student_5	CS	2.5
6	Student_6	EE	2.6
7	Student_7	ME	2.7
8	Student_8	CE	2.8
9	Student_9	BIO	2.9
10	Student_11	EE	3.1
11	Student_12	ME	3.2
12	Student_13	CE	3.3
13	Student_14	BIO	3.4
14	Student_15	CS	3.5
15	Student_16	EE	3.6
16	Student_17	ME	3.7
17	Student_18	CE	3.8
18	Student_19	BIO	3.9
19	Student_21	EE	4.1
20	Student_22	ME	4.2

Refresh Save Cancel Export data 200 200+ 200 row(s) fetched - 0.004s, on 2026-08-12 at 16:32:01

18. 한 번도 수강하지 않은 학생

*cskala_db> Script-2 X

```

ORDER BY
  s.student_id ASC;

-- [종합실습 2 - 18번]
-- 수강 기록이 한 건도 없는 학생을 조회한다.
--
-- NOT EXISTS:
-- 현재 학생과 일치하는 수강 기록이 존재하지 않을 때 TRUE가 된다.
-- 따라서 enroll에 기록이 없는 학생만 결과에 포함된다.
=====

SELECT
  s.student_id AS 학생_ID, -- 학생 번호
  s.name AS 학생명, -- 학생 이름
  s.major AS 전공, -- 학생 전공
  s.gpa AS GPA -- 학생 GPA
FROM lab.student AS s
WHERE NOT EXISTS (
  SELECT 1
  FROM lab.enroll AS e
  WHERE e.student_id = s.student_id
  -- 현재 학생의 수강 기록이 존재하는지 확인한다.
)
ORDER BY
  s.student_id ASC;

```

student 1 X

SELECT s.student_id AS 학생_ID, s.name AS 학생명

123 = 학생_ID	AZ 학생명	AZ 전공	123 gpa
10	Student_10	CS	3
20	Student_20	CS	4
30	Student_30	CS	2
40	Student_40	CS	3
50	Student_50	CS	4
60	Student_60	CS	2
70	Student_70	CS	3
80	Student_80	CS	4
90	Student_90	CS	2
100	Student_100	CS	3
110	Student_110	CS	4
120	Student_120	CS	2
130	Student_130	CS	3
140	Student_140	CS	4
150	Student_150	CS	2
160	Student_160	CS	3
170	Student_170	CS	4
180	Student_180	CS	2
190	Student_190	CS	3
200	Student_200	CS	4

Refresh Save Cancel Export data 200 100 100 row(s) fetched - 0.005s, on 2026-08-12 at 16:33:57

19. HR 학과 학생 일부와의 비교 데모

*cskala_db> Script-2 X

```
-- [종합실습 2 - 19번]
-- HR 전공 학생 981, 985, 990번과 다른 전공 학생의 GPA를 비교한다.
--
-- Self Join:
-- student 테이블을 HR 학생용(hr)과 비교 학생용(other_s)으로
-- 두 번 참조한다.
--
-- JOIN 조건:
-- 1. 서로 같은 학생은 제외한다.
-- 2. 두 학생의 GPA 차이가 0.1 미만인 경우만 연결한다.
--
-- 필터 조건:
-- 1. hr은 HR 전공 학생만 선택한다.
-- 2. other_s는 HR이 아닌 학생만 선택한다.
-- 3. HR 학생은 981, 985, 990번으로 제한한다.
--
=====
SELECT
    hr.name AS HR_학생,      -- 비교 기준이 되는 HR 학생
    hr.gpa AS HR_GPA,       -- HR 학생의 GPA
    other_s.name AS 비교_학생, -- GPA가 비슷한 타 전공 학생
    other_s.gpa AS 비교_GPA, -- 타 전공 학생의 GPA
    other_s.major AS 전공    -- 비교 학생의 전공
FROM lab.student AS hr
INNER JOIN lab.student AS other_s
ON hr.student_id <> other_s.student_id
-- 같은 학생끼리 연결되지 않도록 제외한다.

AND ABS(hr.gpa - other_s.gpa) < 0.1
-- 두 학생의 GPA 차이가 0.1 미만인 경우만 연결한다.

WHERE hr.major = 'HR'
-- 왼쪽은 HR 전공 학생만 선택한다.

AND other_s.major <> 'HR'
-- 오른쪽은 HR이 아닌 학생만 선택한다.

AND hr.student_id IN (981, 985, 990)
-- 지정된 HR 학생 세 명만 비교한다.

ORDER BY
    HR_학생 ASC,
    비교_GPA DESC;
```

student 1 X

SELECT hr.name AS HR_학생, hr.gpa AS HR_GPA, other_s.name AS 비교_학생, other_s.gpa AS 비교_GPA, other_s.major AS 전공

	AZ HR_학생	123 hr_gpa	AZ 비교_학생	123 비교_gpa	AZ 전공
93	Student_990		2 Student_870	2 CS	
94	Student_990		2 Student_900	2 CS	
95	Student_990		2 Student_930	2 CS	
96	Student_990		2 Student_960	2 CS	

Value X

Student_981

20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록

*cskala_db> Script-2 X

```
-- 비교_GPA DESC;
--
-- [종합실습 2 - 20번]
-- 다음 조건 중 하나 이상을 만족하는 학생을 조회한다.
--
-- 조건 1: 전공이 CS인 학생
-- 조건 2: DB 과목을 수강한 학생
--
-- OR:
-- 두 조건 중 하나만 만족해도 결과에 포함한다.
--
-- EXISTS:
-- 현재 학생에게 DB 과목 수강 기록이 존재하는지 확인한다.
-- 여러 수강 기록이 있어도 학생은 한 행만 출력된다.
--
=====
SELECT
    s.student_id AS 학생_ID, -- 학생 번호
    s.name AS 학생명,       -- 학생 이름
    s.major AS 전공,        -- 학생 전공
    s.gpa AS GPA            -- 학생 GPA
FROM lab.student AS s
WHERE s.major = 'CS'
OR EXISTS (
    SELECT 1
    FROM lab.enroll AS e
    WHERE e.student_id = s.student_id
    AND e.course = 'DB'
    -- 현재 학생에게 DB 과목 수강 기록이 있는지 확인한다.
)
ORDER BY
    s.student_id ASC;
```

student 1 X

SELECT s.student_id AS 학생_ID, s.name AS 학생명, s.major AS 전공, s.gpa AS GPA

	123 학생_ID	AZ 학생명	AZ 전공	123 gpa
278	944	Student_944	BIO	3.4
279	945	Student_945	CS	3.5
280	950	Student_950	CS	4
281	955	Student_955	CS	4.5
282	960	Student_960	CS	2
283	963	Student_963	CE	2.3
284	964	Student_964	BIO	2.4
285	965	Student_965	CS	2.5
286	970	Student_970	CS	3
287	975	Student_975	CS	3.5
288	980	Student_980	CS	4
289	985	Student_985	HR	4.5
290	986	Student_986	HR	4.6

Value X

5

21. student 테이블에는 학과(major)와 GPA가 있습니다. 학과별·GPA 구간별 인원을 집계하되, 소계(소계 행)와 총계 행까지 한 번의 쿼리로 아래의 내용이 반영되도록 출력

- GPA를 구간 (3.0 미만 / 3.0~3.5 / 3.5 초과)으로 분류하는 파생 컬럼을 추가
- GROUP BY ROLLUP(major, gpa_tier)로 학과별, 전체 소계를 동시에 조회
- GROUPING(major) 함수로 소계 행에 '전체' 라벨을 붙일 것
- 결과를 major, gpa_tier 순으로 정렬하되 소계 행은 하단에 표시

The screenshot shows a SQL script in a text editor and its results in a table. The script uses a CASE statement to categorize GPA into three tiers: '3.0 미만', '3.0~3.5', and '3.5 초과'. It then uses GROUP BY ROLLUP(major, gpa_tier) to aggregate data by department and GPA tier, with a final GROUP BY ROLLUP(major) to calculate the total for each department. The results table shows the following data:

dept	major	gpa_tier	count
15	EE	3.5 초과	97
16	EE	소계	196
17	HR	3.0 미만	10
18	HR	3.0~3.5	1
19	HR	3.5 초과	9
20	HR	소계	20
21	ME	3.0 미만	66
22	ME	3.0~3.5	33
23	ME	3.5 초과	97
24	ME	소계	196
25	전체	총계	1,000

22. emp 테이블은 CEO(manager_id = NULL) → 매니저(10명) → 개발자(300명)의 3단계 계층 구조입니다. WITH RECURSIVE로 모든 직원의 계층 경로와 깊이를 출력

- WITH RECURSIVE로 CEO에서 시작하는 조직 트리를 탐색
- 각 행에 depth(0=CEO), path 컬럼을 포함 (예: 'CEO > Mgr_2 > Dev_15')
- 매니저별 직속 부하 직원 수를 집계하는 쿼리 별도로 작성 (컬럼명 : direct_reports)

The screenshot shows a SQL script using WITH RECURSIVE to traverse the employee hierarchy. The script starts with the CEO (manager_id = NULL) and recursively finds all subordinates. The results table shows the following data:

emp_id	last_name	manager_id	depth	path
1	CEO	NULL	0	CEO
2	Dev_02	1	1	CEO > Dev_02
3	Dev_12	1	1	CEO > Dev_12
4	Dev_32	1	1	CEO > Dev_32
5	Dev_132	1	1	CEO > Dev_132
6	Dev_132	1	1	CEO > Dev_132
7	Dev_142	1	1	CEO > Dev_142
8	Dev_152	1	1	CEO > Dev_152

22.1 전체 직원의 조직 계층 경로와 깊이 조회

The screenshot shows a SQL script using a subquery to count the number of direct reports for each manager. The results table shows the following data:

emp_id	last_name	direct_reports
1	CEO	300
2	Mgr_2	30
3	Mgr_3	30
4	Mgr_4	30
5	Mgr_5	30
6	Mgr_6	30
7	Mgr_7	30
8	Mgr_8	30
9	Mgr_9	30
10	Mgr_10	30
11	Mgr_11	0

22.2 매니저별 직속 부하 직원 수 집계

23. 각 학과(major)별로 GPA가 높은 순서로 순위를 매기고, 상위 3명씩만 추출하는 쿼리를 Window Function으로 작성 (서브쿼리 방식과 CTE 방식 모두 작성)

- 학과 내 순위를 계산 ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC)
- GPA가 동일한 경우 student_id 오름차순을 2차 기준으로 사용
- RANK()와 DENSE_RANK()를 함께 계산해 동점 처리 방식 차이를 비교
- 결과에 전체 학과별 학생 수(total_in_major)도 COUNT() OVER(PARTITION BY major)로 추가

Query: `SELECT ranked_student_id AS 학생_ID, 학생명, 학과명, gpa, RANK() OVER (PARTITION BY 학과명 ORDER BY gpa DESC) AS 순위, COUNT(*) OVER (PARTITION BY 학과명) AS 전체_학생수 FROM scored_enrollments AS se`

학생_ID	학생명	학과명	gpa	순위	전체_학생수
1	Student_29	BIO	4.9	1	1
2	Student_59	BIO	4.9	2	1
3	Student_89	BIO	4.9	3	1
4	Student_29	CE	4.8	1	1
5	Student_58	CE	4.8	2	1
6	Student_88	CE	4.8	3	1
7	Student_25	CS	4.5	1	1
8	Student_55	CS	4.5	2	1
9	Student_85	CS	4.5	3	1
10	Student_26	EE	4.6	1	1
11	Student_56	EE	4.6	2	1
12	Student_86	EE	4.6	3	1

Query: `WITH ranked_students AS (SELECT student_id, 학생명, 학과명, gpa, RANK() OVER (PARTITION BY 학과명 ORDER BY gpa DESC) AS 순위, COUNT(*) OVER (PARTITION BY 학과명) AS 전체_학생수 FROM scored_enrollments AS se)`

학생_ID	학생명	학과명	gpa	순위	전체_학생수
1	Student_29	BIO	4.9	1	1
2	Student_59	BIO	4.9	2	1
3	Student_89	BIO	4.9	3	1
4	Student_29	CE	4.8	1	1
5	Student_58	CE	4.8	2	1
6	Student_88	CE	4.8	3	1
7	Student_25	CS	4.5	1	1
8	Student_55	CS	4.5	2	1
9	Student_85	CS	4.5	3	1
10	Student_26	EE	4.6	1	1
11	Student_56	EE	4.6	2	1
12	Student_86	EE	4.6	3	1

22.1 FROM 안에 서브쿼리를 작성

22.1 WITH 절에서 CTE로 먼저 작성

24. enroll 테이블을 student_id 기준으로 정렬하고, 각 학생의 이전 수강 과목 대비 성적 변화를 LAG()로 계산 (성적은 A=4, B=3, C=2, D=1 점수로 변환)

- grade를 숫자 점수(A=4, B=3, C=2, D=1)로 변환하는 CASE 식을 작성
- LAG(score) OVER (PARTITION BY student_id ORDER BY course)로 이전 과목 점수를 가져올 것
- 현재 점수 - 이전 점수를 diff 컬럼으로 추가하고, 상승/유지/하락을 텍스트로 표시
- 학생별 최고점과 최저점의 차이(score_range)를 Window Function으로 계산

Query: `WITH scored_enrollments AS (SELECT student_id, e.course, e.grade, CASE e.grade WHEN 'A' THEN 4 WHEN 'B' THEN 3 WHEN 'C' THEN 2 WHEN 'D' THEN 1 ELSE NULL END AS score FROM lab.enroll AS e), analyzed_enrollments AS (SELECT student_id, se.course, se.grade, se.score, LAG(se.score) OVER (PARTITION BY se.student_id ORDER BY se.course ASC) AS prev_score, MAX(se.score) OVER (PARTITION BY se.student_id) AS max_score, MIN(se.score) OVER (PARTITION BY se.student_id) AS min_score FROM scored_enrollments AS se)`

학생_ID	학과명	이전_점수	현재_점수	점수_차이	성적_변화	학생별_점수_범위
2287	994 Course_17	A	4	1	3	상승
2288	995 Course_17	A	4	[NULL]	[NULL]	비교 없음
2289	995 Course_18	B	3	4	-1	하락
2290	995 Course_19	C	2	3	-1	하락
2291	996 Course_18	B	3	[NULL]	[NULL]	비교 없음
2292	996 Course_19	C	2	3	-1	하락
2293	997 Course_1	A	4	[NULL]	[NULL]	비교 없음
2294	997 Course_19	C	2	4	-2	하락
2295	997 Course_20	D	1	2	-1	하락
2296	998 Course_1	A	4	[NULL]	[NULL]	비교 없음
2297	998 Course_20	D	1	4	-3	하락
2298	999 Course_1	A	4	[NULL]	[NULL]	비교 없음
2299	999 Course_2	B	3	4	-1	하락
2300	999 Course_3	C	2	3	-1	하락
2301	1001 AI	A	4	[NULL]	[NULL]	비교 없음
2302	1010 ML	B	3	[NULL]	[NULL]	비교 없음

Query: `MIN(se.score) OVER (PARTITION BY se.student_id AS score_range FROM scored_enrollments AS se) SELECT student_id, se.course, se.grade, se.score, se.score - se.prev_score AS diff, CASE WHEN se.score > se.prev_score THEN '상' WHEN se.score < se.prev_score THEN '하' WHEN se.score = se.prev_score THEN '유' ELSE '미' END AS 성적_변화, se.score - se.min_score AS 학생별_점수_범위 FROM analyzed_enrollments AS ae ORDER BY ae.student_id ASC, ae.course ASC`

학생_ID	학과명	A2 문자_성적	A3 현재_점수	A4 이전_점수	A5 점수_차이	A6 성적_변화	A7 학생별_점수_범위
2287	994 Course_17	A	4	1	3	상승	
2288	995 Course_17	A	4	[NULL]	[NULL]	비교 없음	
2289	995 Course_18	B	3	4	-1	하락	
2290	995 Course_19	C	2	3	-1	하락	
2291	996 Course_18	B	3	[NULL]	[NULL]	비교 없음	
2292	996 Course_19	C	2	3	-1	하락	
2293	997 Course_1	A	4	[NULL]	[NULL]	비교 없음	
2294	997 Course_19	C	2	4	-2	하락	
2295	997 Course_20	D	1	2	-1	하락	
2296	998 Course_1	A	4	[NULL]	[NULL]	비교 없음	
2297	998 Course_20	D	1	4	-3	하락	
2298	999 Course_1	A	4	[NULL]	[NULL]	비교 없음	
2299	999 Course_2	B	3	4	-1	하락	
2300	999 Course_3	C	2	3	-1	하락	
2301	1001 AI	A	4	[NULL]	[NULL]	비교 없음	
2302	1010 ML	B	3	[NULL]	[NULL]	비교 없음	

- enroll 테이블의 모든 수강 기록을 분석하기 때문에 학생 정보가 없는 1001번과 1010번 수강도 포함됨

25. orders 테이블의 주문을 order_id 순으로 정렬하여 누적 주문금액과 3개 주문 이동평균 계산 (ROWS BETWEEN 사용)

- SUM(amount) OVER (ORDER BY order_id ROWS BETWEEN UNBOUNDED PRECEDING

AND CURRENT ROW)로 누적합 계산

- AVG(amount) OVER (ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)로 3개 이동평균 계산

- customer_id별로 PARTITION을 나눠 고객별 누적 구매금액도 함께 계산

- 누적합이 전체 합 50%를 초음하는 첫 번째 order_id를 찾는 쿼리를 작성

o=kslab_db-Script-2 X

SQL Script-2 X

1. 주문금액

2. 3개 주문 이동평균

3. 고객별 누적 구매금액

SELECT

o.order_id AS 주문_ID,

o.customer_id AS 고객_ID,

o.amount AS 주문금액,

FROM orders o

ORDER BY o.order_id

--- 고객별 누적 주문금액 (PARTITION BY customer_id)

--- 누적 주문금액 (SUM(amount) OVER (ORDER BY o.order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW))

--- 3개 주문 이동평균 (AVG(amount) OVER (ORDER BY o.order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW))

--- 누적 주문금액 (SUM(amount) OVER (ORDER BY o.order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW))

--- 3개 주문 이동평균 (AVG(amount) OVER (ORDER BY o.order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW))

SELECT o.order_id AS 주문_ID,

o.customer_id AS 고객_ID,

o.amount AS 주문금액,

SUM(amount) OVER (ORDER BY o.order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS 누적주문금액,

AVG(amount) OVER (ORDER BY o.order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS 3개주문이동평균,

SUM(amount) OVER (PARTITION BY o.customer_id ORDER BY o.order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS 고객별누적주문금액

FROM lab.orders o

ORDER BY o.order_id ASC;

orders 1 X

SELECT o.order_id AS 주문_ID, o.customer_id AS 고객_ID, Enter a SQL expression to filter results to show CUST=Space

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

o

SQL Script-2 X

```

-- [문장] 2 - 25.11
-- 고객 주문_id 순으로 정렬하여,
-- 1. 누적 주문금액
-- 2. 3개 주문 이동평균
-- 3. 고객별 누적 구매금액

WITH order_amounts AS (
    SELECT o.order_id AS 주문_id,
           o.customer_id AS 고객_id,
           o.amount AS 주문금액,
           SUM(amount) OVER (ORDER BY o.order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS 누적주문금액,
           AVG(amount) OVER (ORDER BY o.order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS 3개주문이동평균,
           SUM(amount) OVER (PARTITION BY o.customer_id ORDER BY o.order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS 고객별누적주문금액
    FROM lab.orders o
)

SELECT o.order_id AS 주문_id,
       o.customer_id AS 고객_id,
       o.amount AS 주문금액,
       SUM(amount) OVER (ORDER BY o.order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS 누적주문금액,
       AVG(amount) OVER (ORDER BY o.order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS 3개주문이동평균,
       SUM(amount) OVER (PARTITION BY o.customer_id ORDER BY o.order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS 고객별누적주문금액
FROM order_amounts o
ORDER BY o.order_id ASC;

```

주문_id	고객_id	주문금액	누적주문금액	3개주문이동평균	고객별누적주문금액
1503	4	4,544.03	3,686,585.56	7,365.985	50.05

[별첨] 실습 데이터베이스 ERD

